

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 15

Balanced Trees

Last Lecture

Transform and conquer:

- Pre-sort

Last Lecture

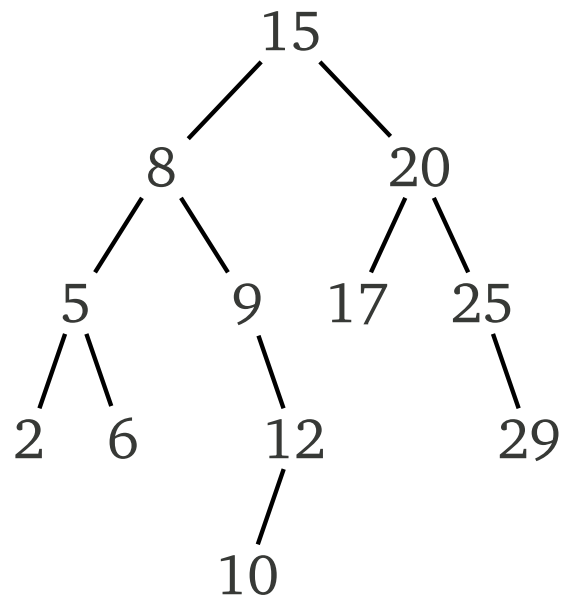
Transform and conquer:

- Pre-sort
- Binary Search Trees

Last Lecture

Transform and conquer:

- Pre-sort
- Binary Search Trees



Reasonably-balanced Binary Search Trees

Instance simplification: Self-balancing trees

- AVL trees
- Red-black trees
- Splay trees

Representational changes

- 2–3 trees
- 2–3–4 trees
- B-trees

AVL Trees

Name

Adelson-Velsky and Landis (1962)

Definition: Balance Factor

For a BST its balance factor is the difference between the height of its left sub-tree and that of its right sub-tree.

Single tree node with one value and no children (2 NULL pointers): height $= 0$

No tree / empty tree (just a NULL pointer): height $= -1$

AVL Trees

Name

Adelson-Velsky and Landis (1962)

Definition: Balance Factor

For a BST its balance factor is the difference between the height of its left sub-tree and that of its right sub-tree.

Definition: AVL Tree

An AVL tree is a BST in which the balance factor is -1, 0, or 1, for every sub-tree.

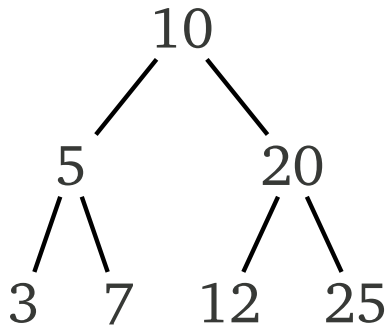
Single tree node with one value and no children (2 NULL pointers): height $= 0$

No tree / empty tree (just a NULL pointer): height $= -1$

AVL Trees: Examples and Counter-Examples



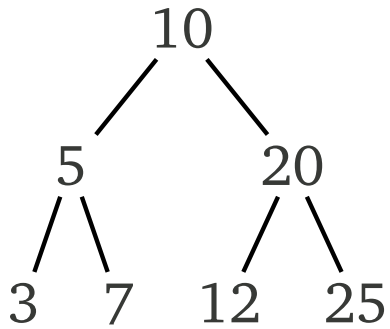
Which of these are AVL trees?

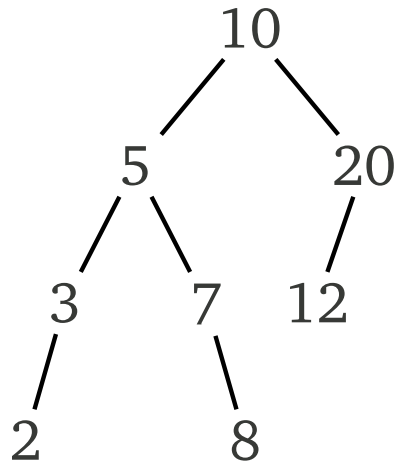


AVL Trees: Examples and Counter-Examples



Which of these are AVL trees?

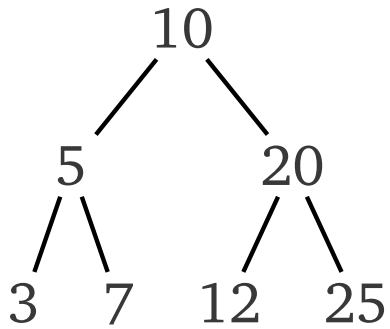


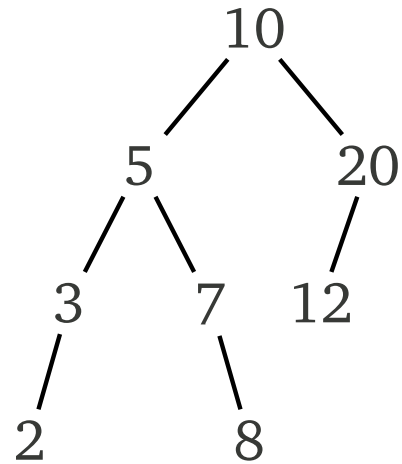


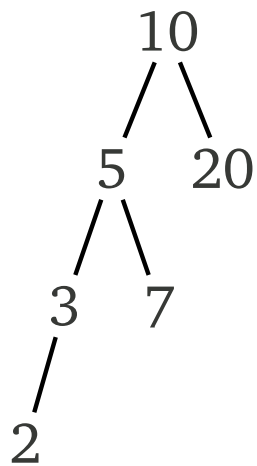
AVL Trees: Examples and Counter-Examples



Which of these are AVL trees?



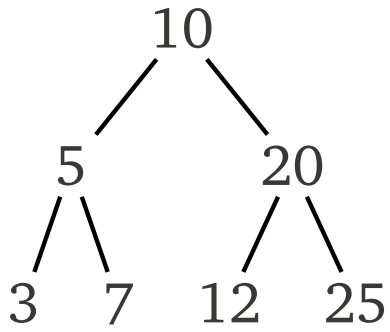


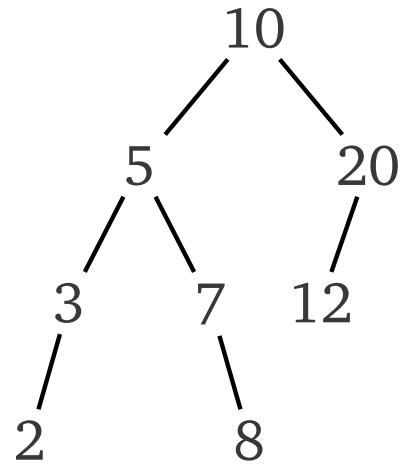


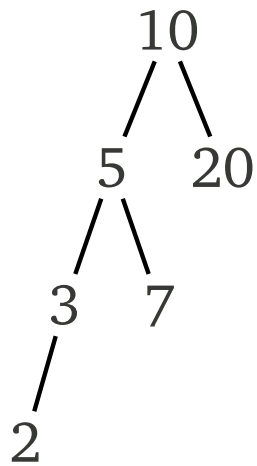
AVL Trees: Examples and Counter-Examples

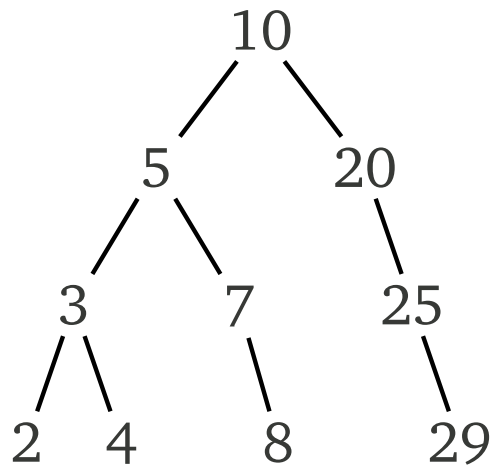


Which of these are AVL trees?









Building an AVL Tree

Insertion

- At the fringe of the tree
- Tree can lose balance
  (some nodes get balance factors of 2 or -2)

Building an AVL Tree

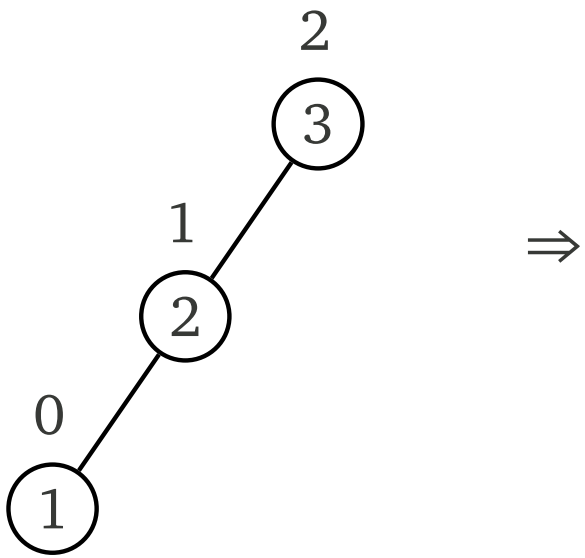
Insertion

- At the fringe of the tree
- Tree can lose balance
  (some nodes get balance factors of 2 or -2)

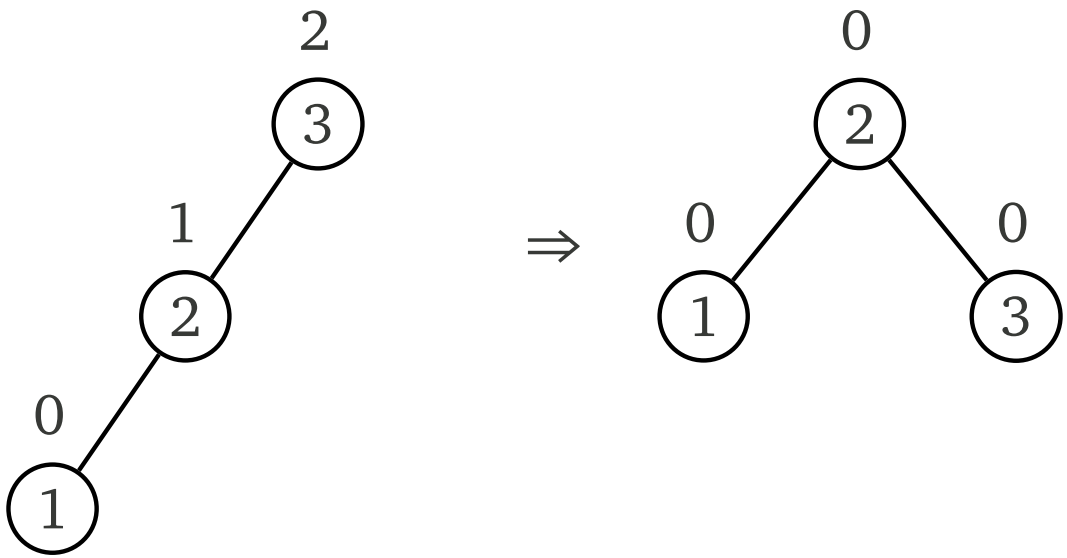
Regain Balance

- local rotations

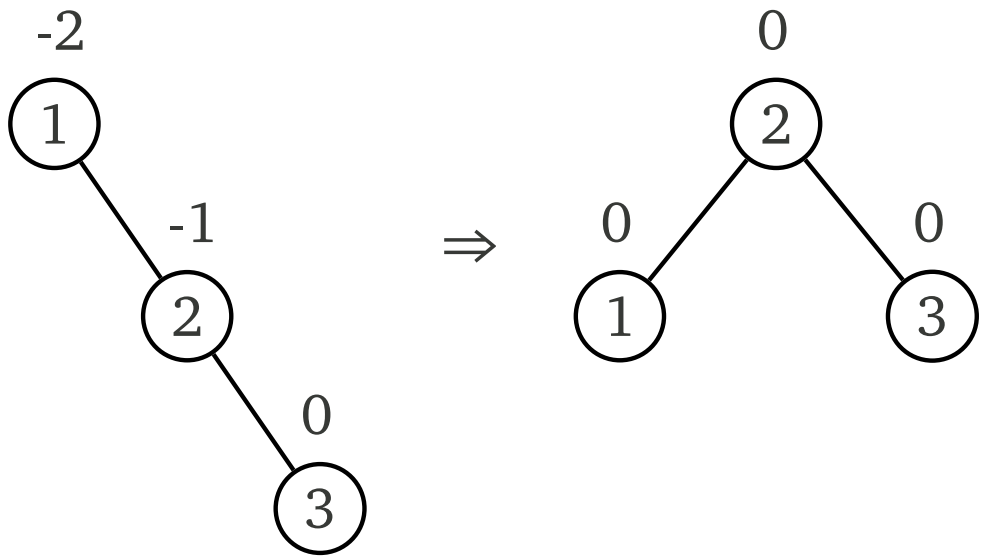
AVL Trees: R-Rotation



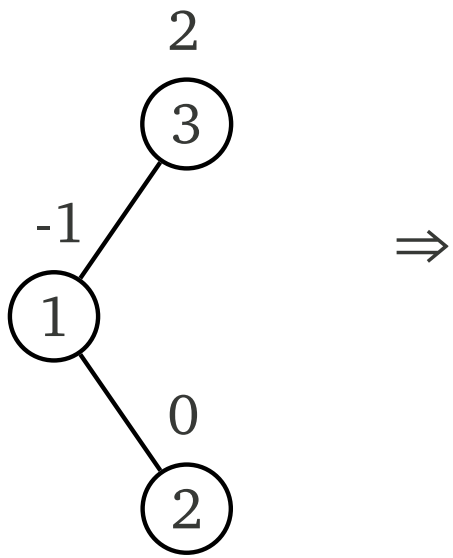
AVL Trees: R-Rotation



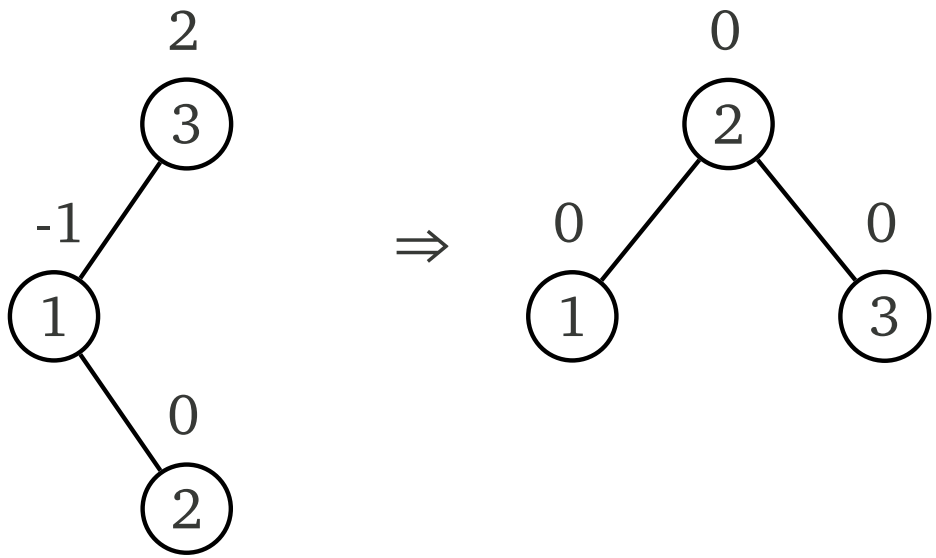
AVL Trees: L-Rotation



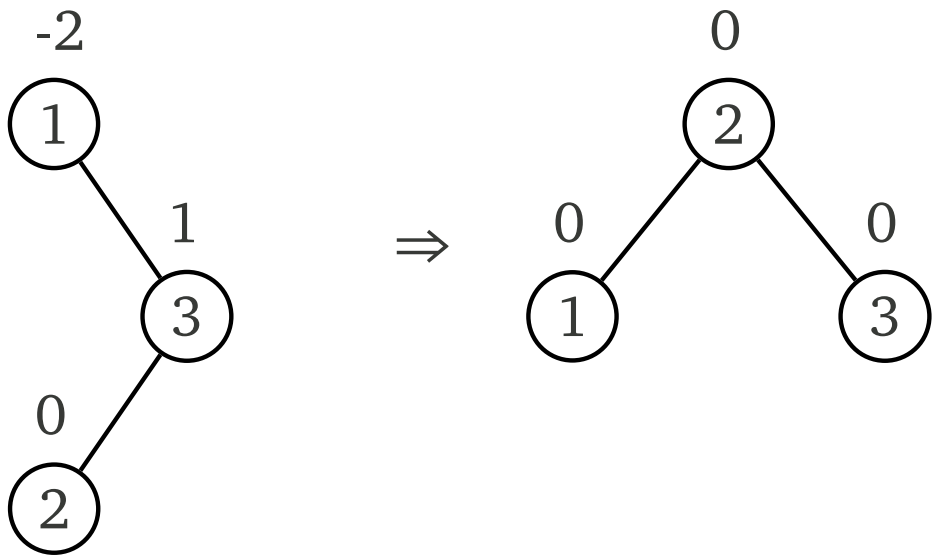
AVL Trees: LR-Rotation



AVL Trees: LR-Rotation

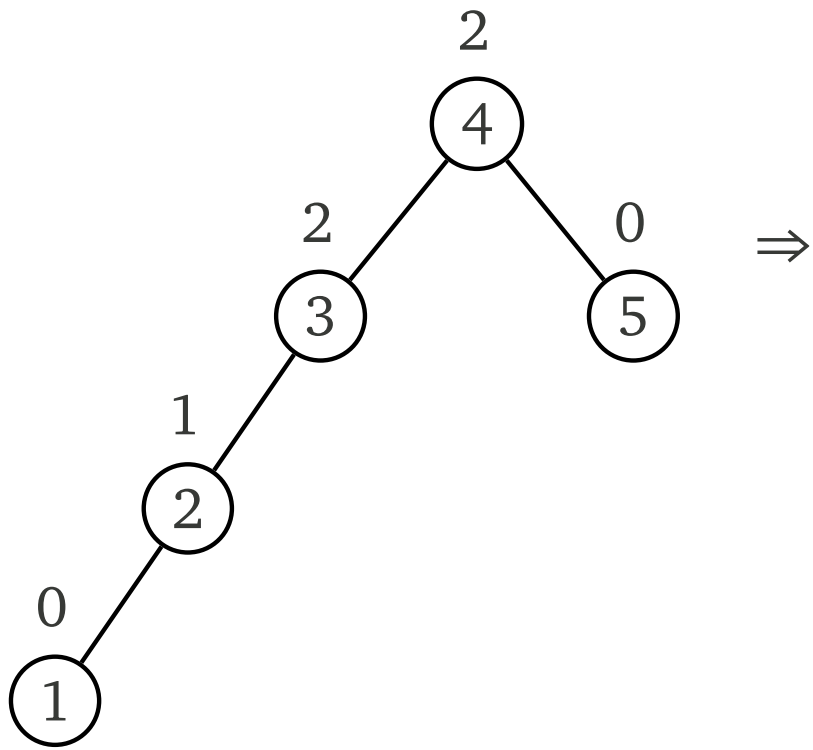


AVL Trees: RL-Rotation



AVL Trees: Where to Perform the Rotation

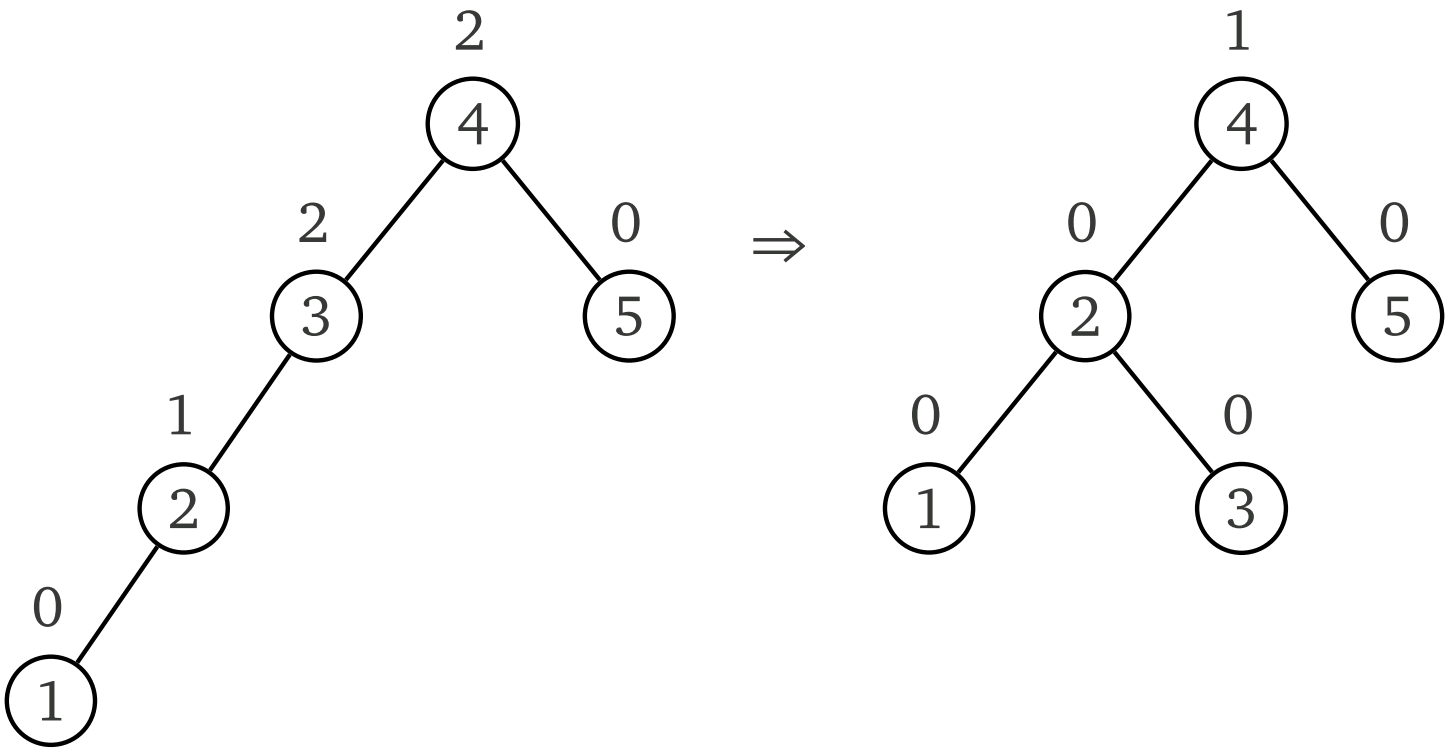
Along an unbalanced path, we may have several nodes with balance factor 2 (or -2):



It is always the lowest unbalanced subtree that is re-balanced.

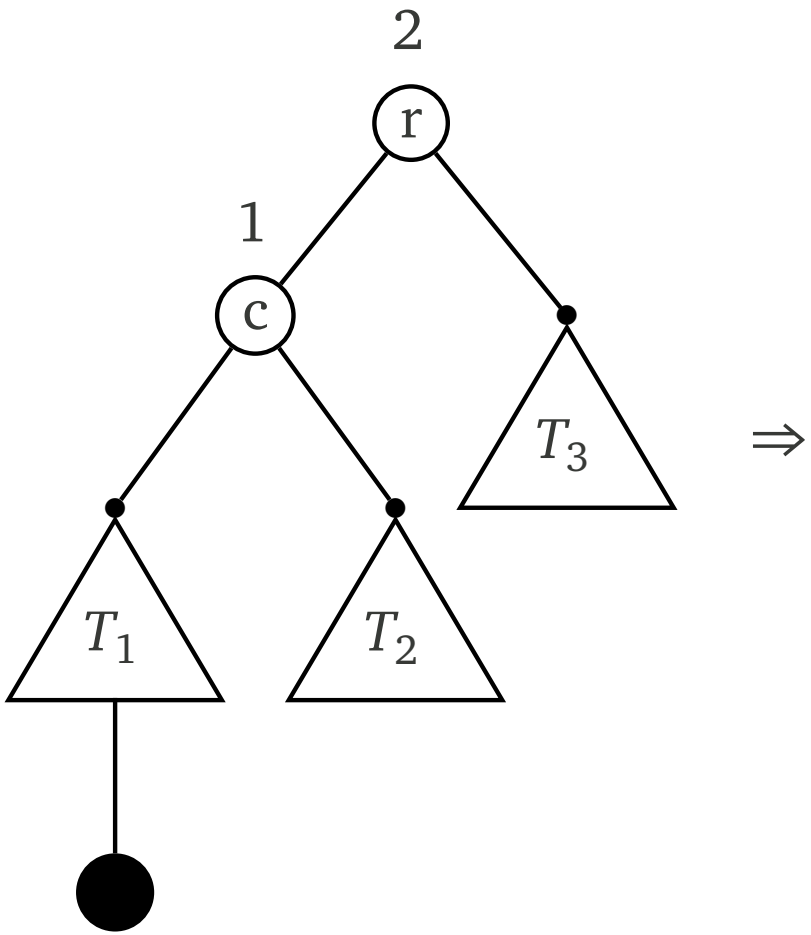
AVL Trees: Where to Perform the Rotation

Along an unbalanced path, we may have several nodes with balance factor 2 (or -2):

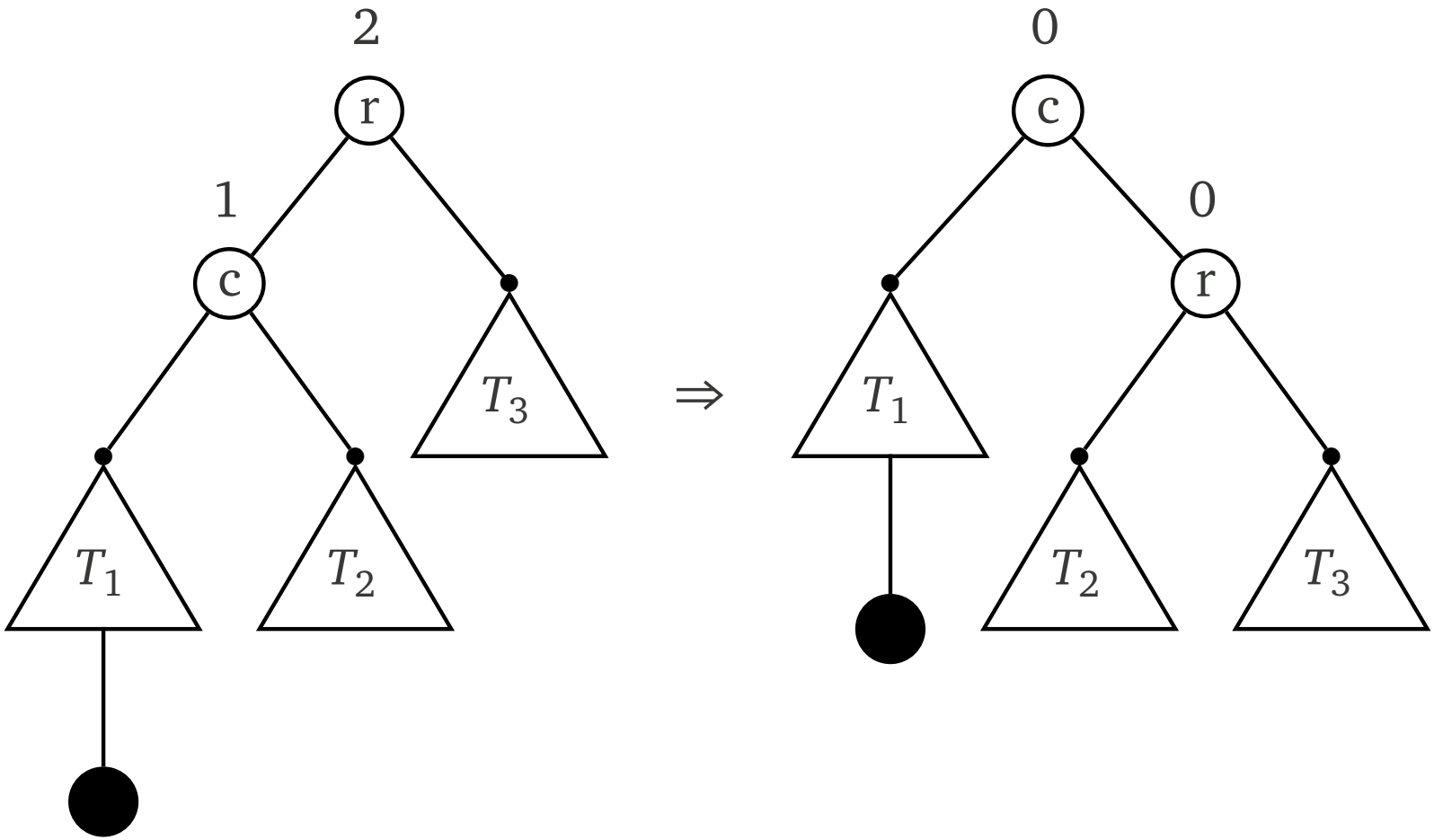


It is always the lowest unbalanced subtree that is re-balanced.

AVL Trees: The Single Rotation, Generally

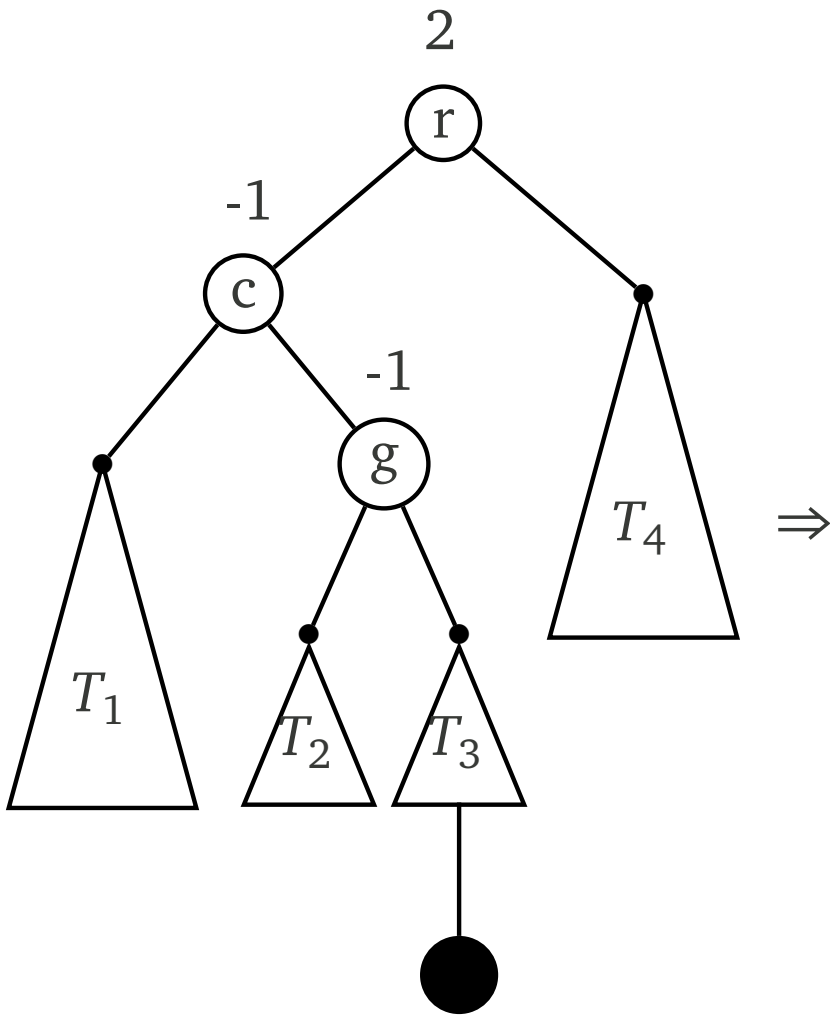


AVL Trees: The Single Rotation, Generally

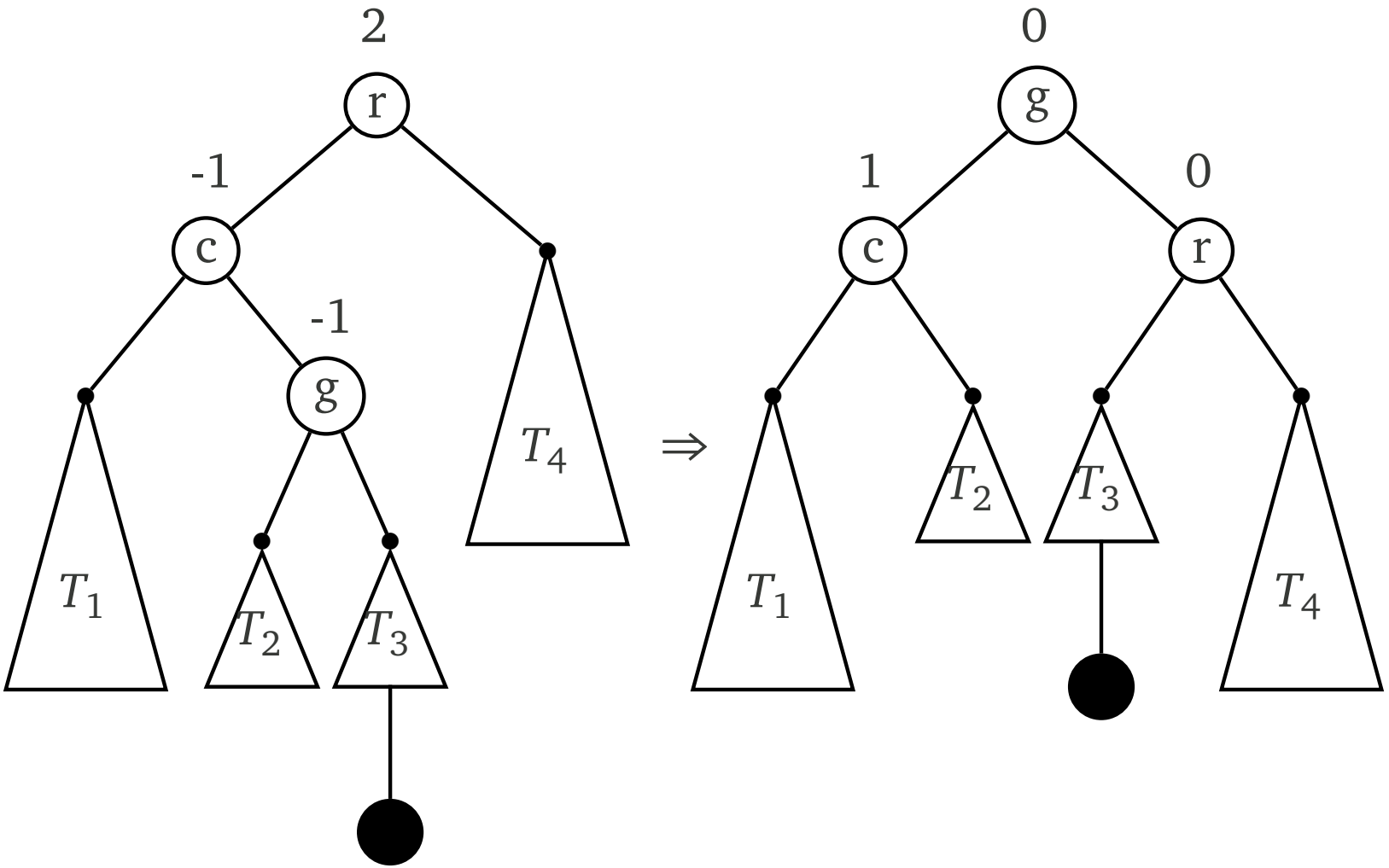


This shows an R-rotation; an L-rotation is similar.

AVL Trees: The Double Rotation, Generally



AVL Trees: The Double Rotation, Generally



This shows an LR-rotation; an RL-rotation is similar.



Properties of AVL Trees

- Thanks to “balance”: depth of an AVL with n nodes is $\Theta(\log n)$.
- Depth close to $\log_2 n$ for random data

Properties of AVL Trees

- Thanks to "balance": depth of an AVL with n nodes is $\Theta(\log n)$.
- Depth close to $\log_2 n$ for random data
- Insertion $\Theta(\log n)$
- Deletion $\Theta(\log n)$ (harder to implement)

Other Kinds of Balanced Trees

A red-black tree is a BSTs with a slightly different concept of "balanced".

Other Kinds of Balanced Trees

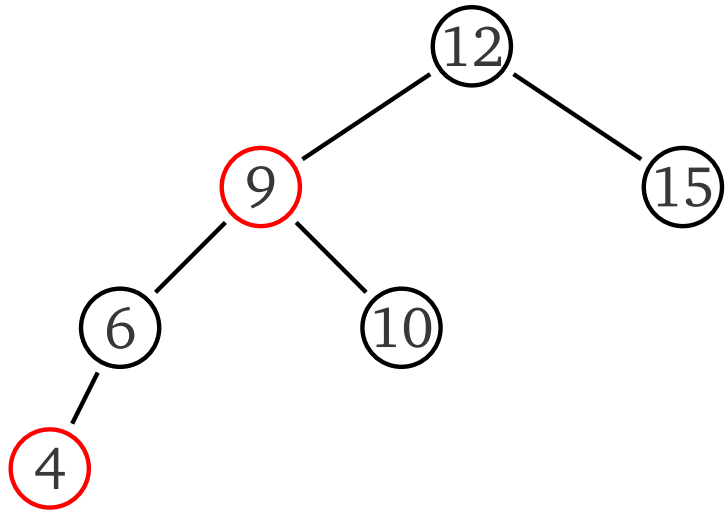
A red-black tree is a BSTs with a slightly different concept of "balanced". Its nodes are coloured red or black, so that

1. No red node has a red child.
2. Every path from the root to the fringe has the same number of black nodes.

Other Kinds of Balanced Trees

A red-black tree is a BSTs with a slightly different concept of "balanced". Its nodes are coloured red or black, so that

1. No red node has a red child.
2. Every path from the root to the fringe has the same number of black nodes.

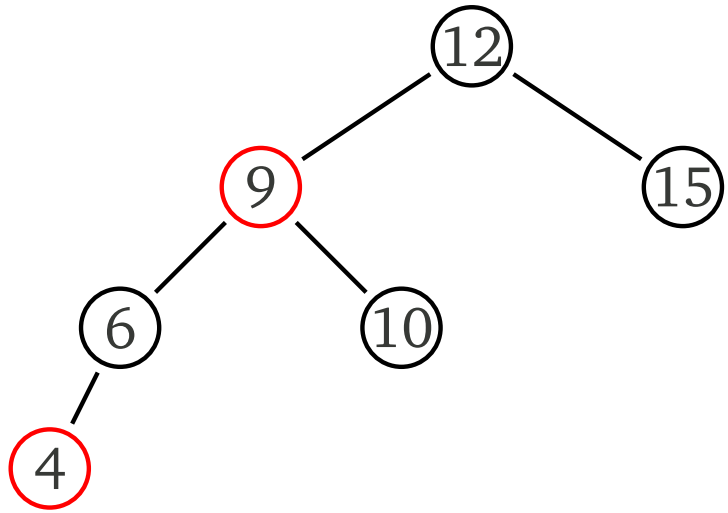


A worst-case red-black tree (the longest path is twice as long as the shortest path).

Other Kinds of Balanced Trees

A red-black tree is a BSTs with a slightly different concept of "balanced". Its nodes are coloured red or black, so that

1. No red node has a red child.
2. Every path from the root to the fringe has the same number of black nodes.



A worst-case red-black tree (the longest path is twice as long as the shortest path).

A splay tree is a BST which is not only self-adjusting, but also adaptive. Frequently accessed items are brought closer to the root, so their access becomes cheaper.

2–3 Trees

Idea

- John Hopcroft, 1970
- Store more items in a tree node

If single item

2-node, has two children

If two items

3-node, three children

2–3 Trees

Idea

- John Hopcroft, 1970
- Store more items in a tree node

If single item

2-node, has two children

If two items

3-node, three children

Main property: perfectly balanced

All leaves are at the same level

2–3 Trees

Idea

- John Hopcroft, 1970
- Store more items in a tree node

If single item

2-node, has two children

If two items

3-node, three children

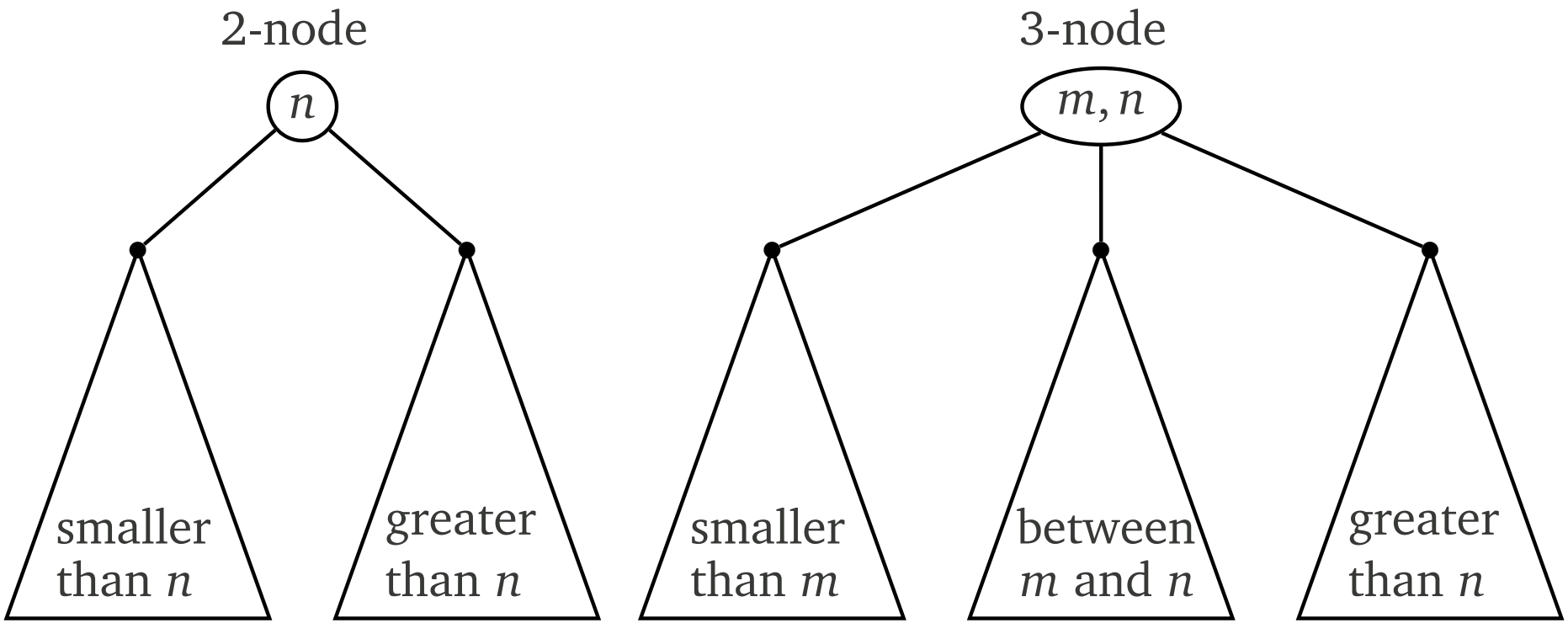
Main property: perfectly balanced

All leaves are at the same level

2–3–4 trees

also allow 4-nodes

2-Nodes and 3-Nodes



Insertion in a 2–3 Tree

To insert a key k, pretend that we are searching for k.

Insertion in a 2–3 Tree

To insert a key k, pretend that we are searching for k.

At correct leaf node: if 2-node, k can be inserted.

Insertion in a 2–3 Tree

To insert a key k, pretend that we are searching for k.

At correct leaf node: if 2-node, k can be inserted.

Otherwise 3-node, and the two inhabitants. Temporarily include k. In sorted order call them k_1, k_2, and k_3.

Insertion in a 2–3 Tree

To insert a key k, pretend that we are searching for k.

At correct leaf node: if 2-node, k can be inserted.

Otherwise 3-node, and the two inhabitants. Temporarily include k. In sorted order call them k_1, k_2, and k_3.

We now split the node, so that k_1 and k_3 form their own individual 2-nodes. The middle key, k_2 is promoted to the parent node.

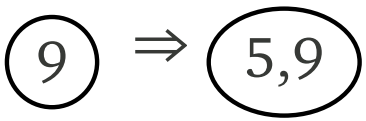
The promotion may cause parent node to overflow:
If so split it the same way.

Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7

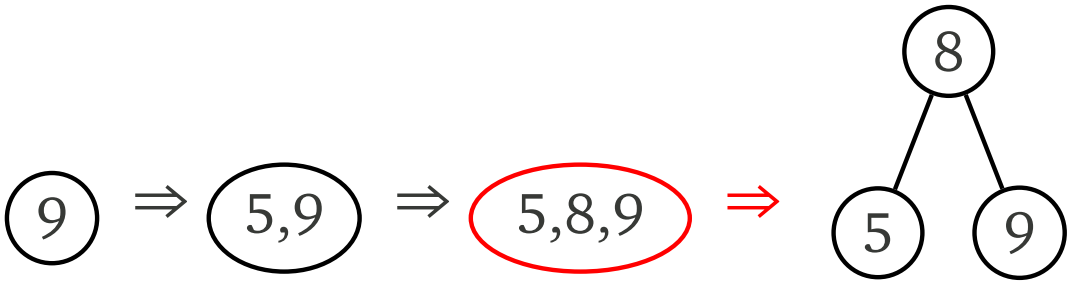
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



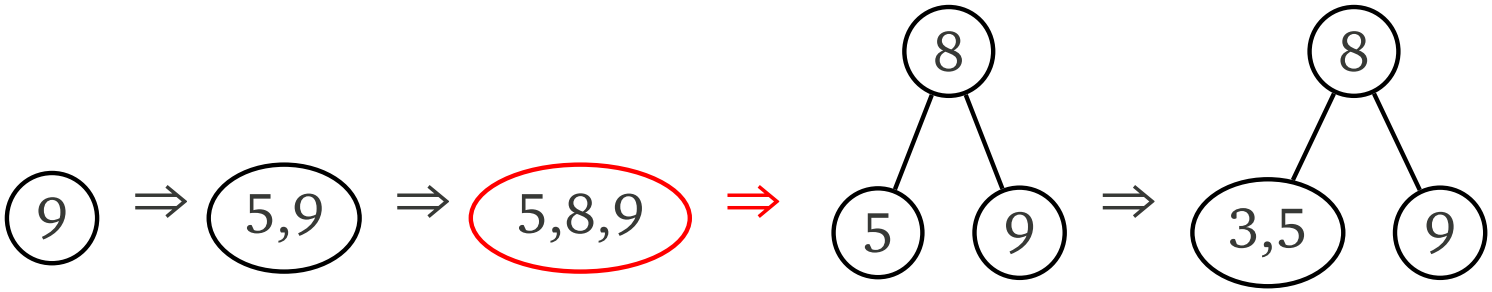
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



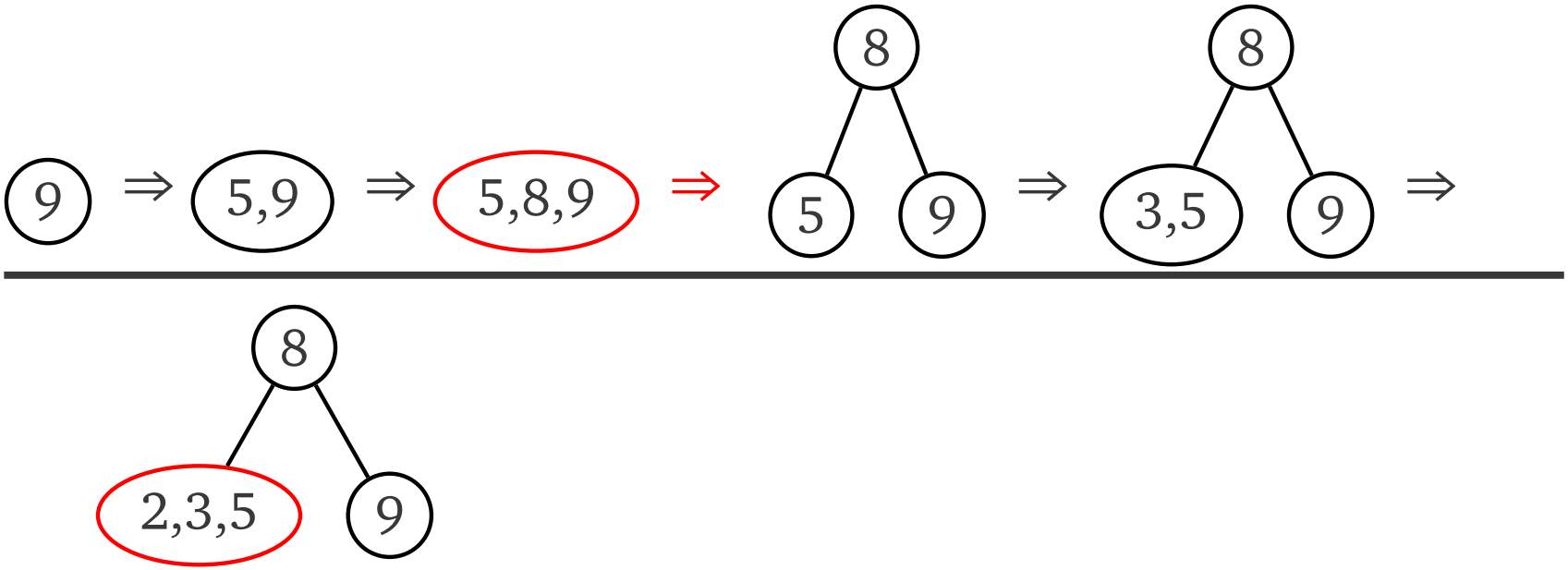
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



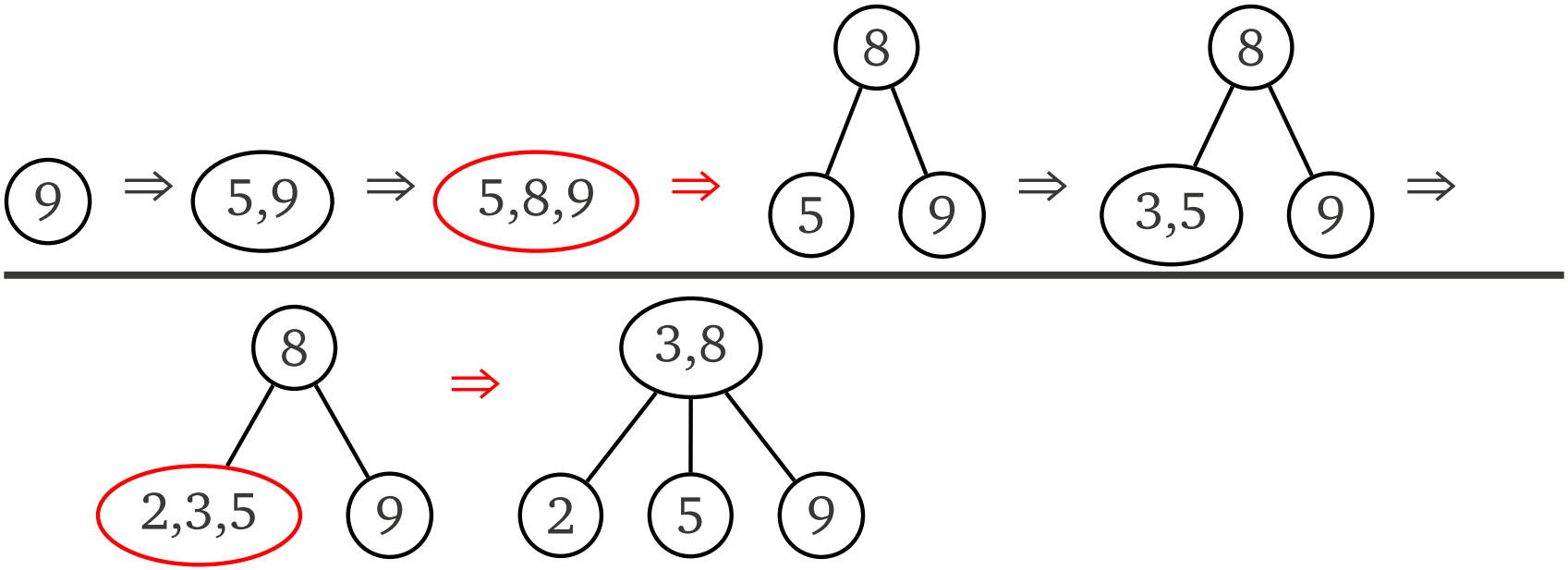
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



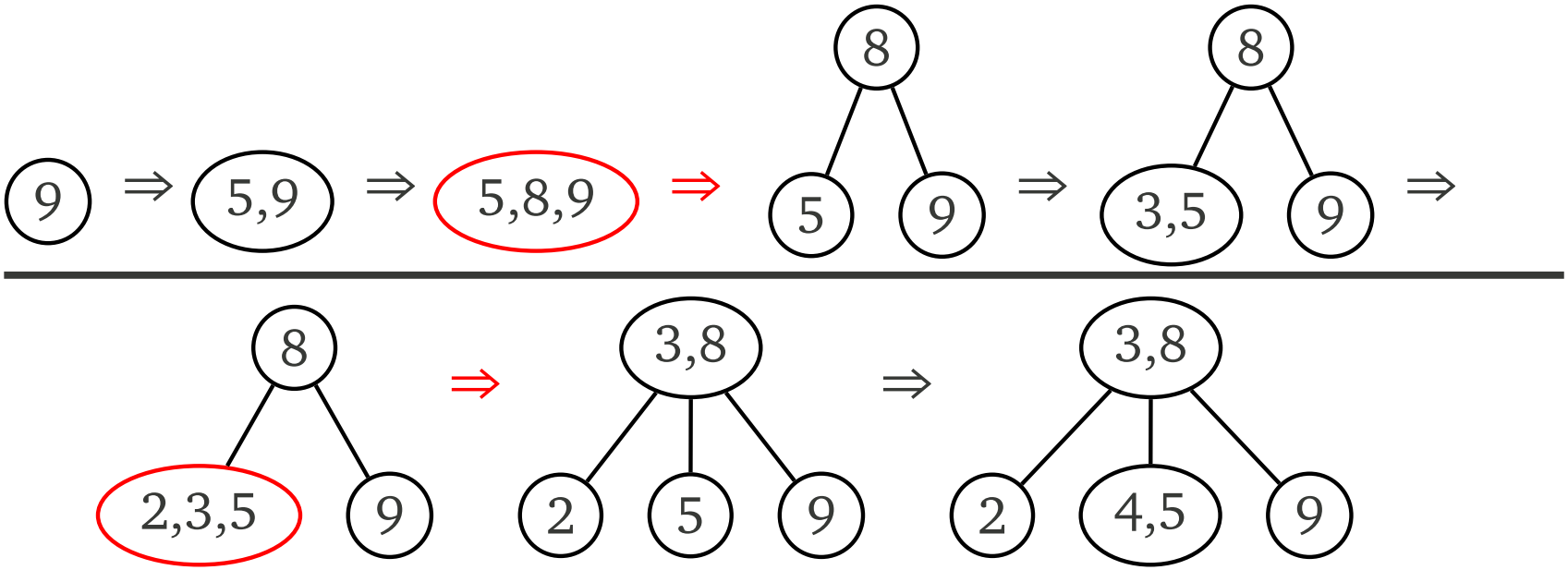
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



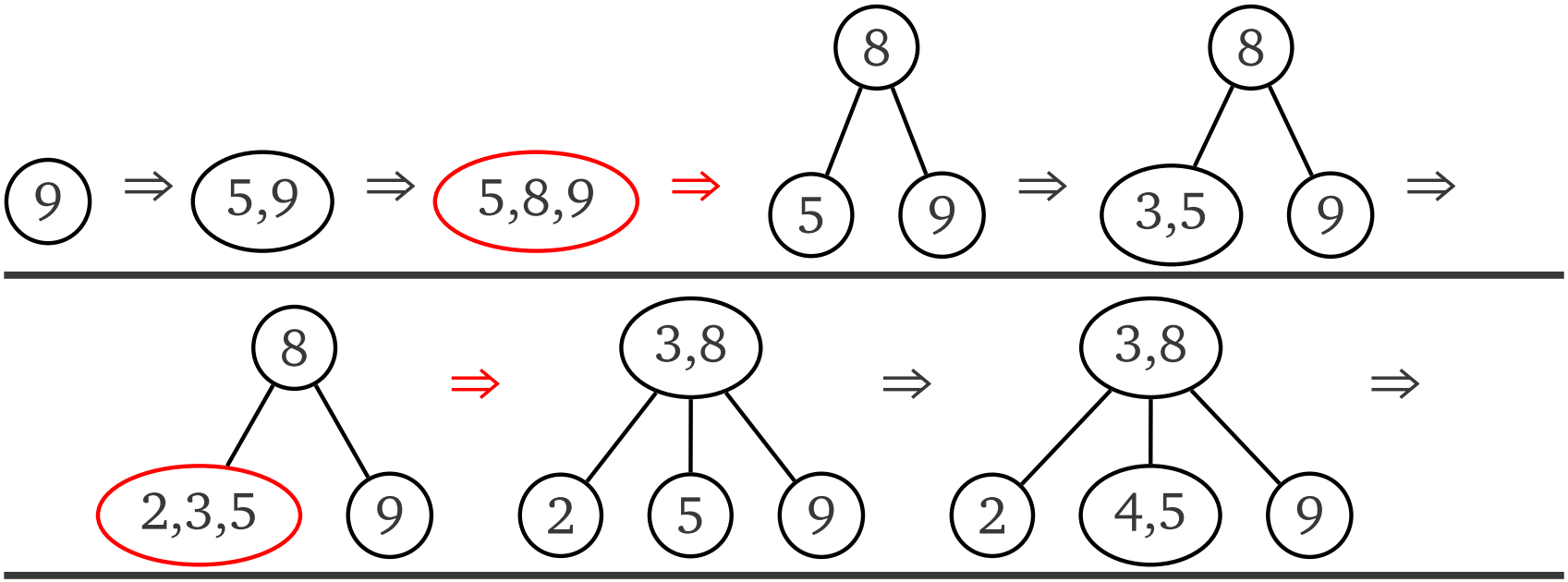
Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7

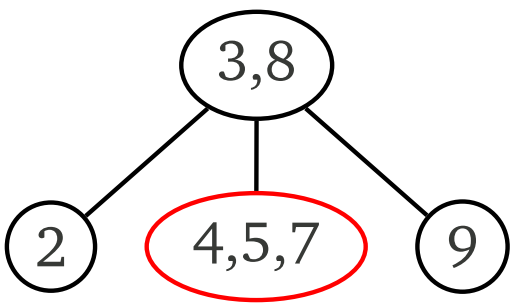


Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7

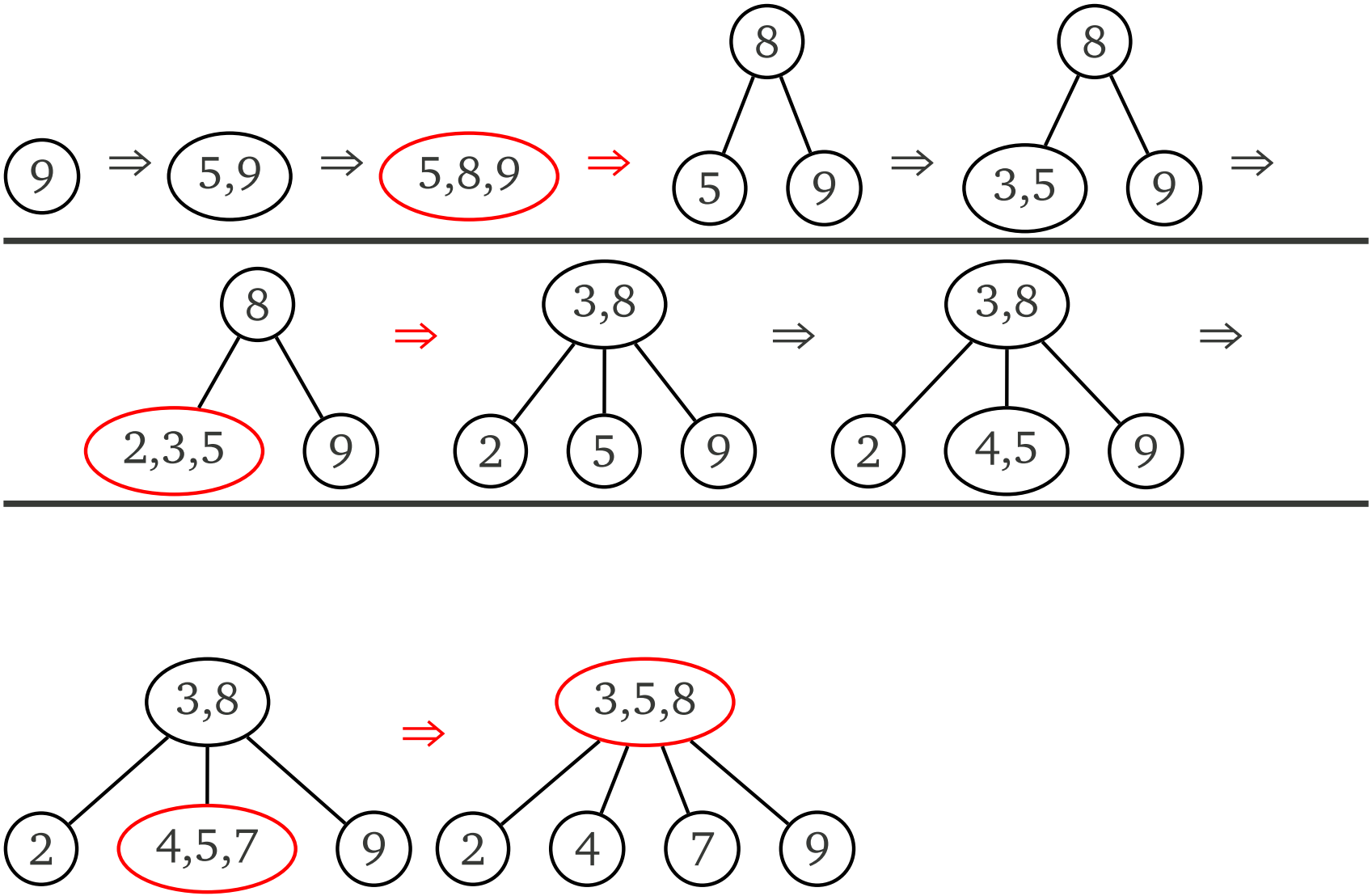


Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7

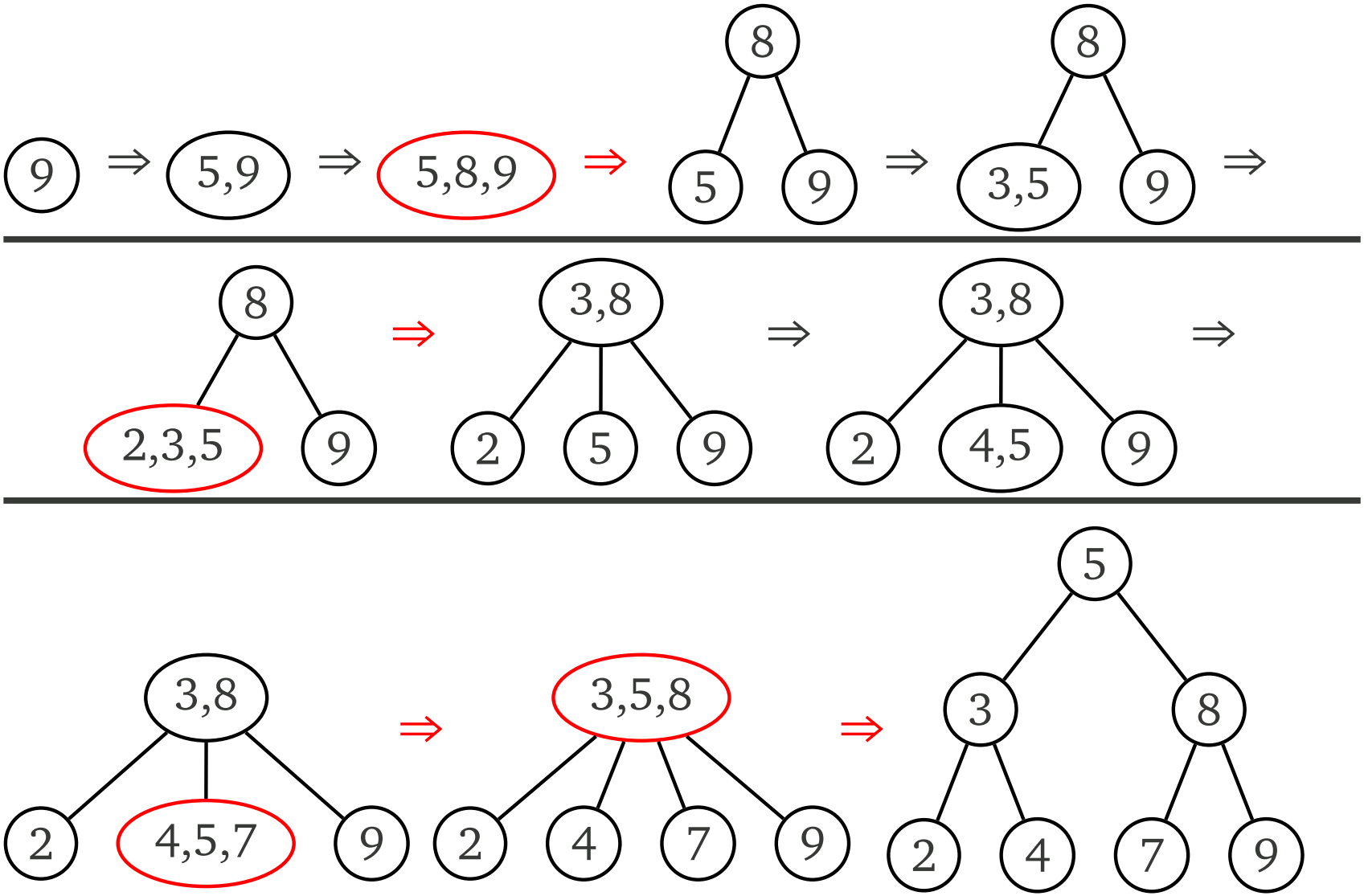




Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



Example: Build a 2–3 Tree from 9, 5, 8, 3, 2, 4, 7



Exercise: 2–3 Tree Construction

Build the 2–3 tree that results from inserting these keys, in the given order, into an initially empty tree:

C, O, M, P, U, T, I, N, G



(Assume: $C < G < I < M < N < O < P < T < U$)

Exercise: 2–3 Tree Construction

Build the 2–3 tree that results from inserting these keys, in the given order, into an initially empty tree:

C, O, M, P, U, T, I, N, G



(Assume: $C < G < I < M < N < O < P < T < U$)

When do we increase the size of the tree?

2–3 Tree Search Time Analysis

2–3 Tree Search Time Analysis

Worst case: all nodes are 2-nodes

Best case: all nodes are 3-nodes

2–3 Tree Search Time Analysis

Worst case: all nodes are 2-nodes

The relation between the number n of nodes and the height h is:

$$n = 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$$

That is, $\log_2(n+1) = h + 1$.

Best case: all nodes are 3-nodes

2–3 Tree Search Time Analysis

Worst case: all nodes are 2-nodes

The relation between the number n of nodes and the height h is:

$$n = 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$$

That is, $\log_2(n+1) = h + 1$.

Best case: all nodes are 3-nodes

$$n = 2 + 2 \times 3 + 2 \times 3^2 + \ldots + 2 \times 3^h = 3^{h+1} - 1$$

That is, $\log_3(n+1) = h + 1$.

Useful formula: $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$, for $a \neq 1$.

2–3 Tree Search Time Analysis

Worst case: all nodes are 2-nodes

The relation between the number n of nodes and the height h is:

$$n = 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$$

That is, $\log_2(n+1) = h + 1$.

Best case: all nodes are 3-nodes

$$n = 2 + 2 \times 3 + 2 \times 3^2 + \ldots + 2 \times 3^h = 3^{h+1} - 1$$

That is, $\log_3(n+1) = h + 1$.

Hence we have $\log_3(n+1) - 1 \leq h \leq \log_2(n+1) - 1$.

Useful formula: $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$, for $a \neq 1$.

Overview

Summary

Balanced trees

Next

Space/Time Trade-offs.